

Supplementary Material

Benchmarking Comparative Single-Cell Analysis of CMV Status
Sebastian Fay Marie Schygulla Gregor Habitzreither

S1 Reference study, dataset and preprocessing

S1.1 The reference study and the implemented comparison

CellCNN learns a sample representation from labelled bags of cells [1]. Its shared filters can respond to uncommon marker combinations; pooling prevents a small informative subset from being averaged indiscriminately into the entire sample. The paper studies several settings, including stimulated immune cells, survival-associated subsets, CMV-associated NK populations and artificial leukaemia spike-ins. These experiments have different endpoints and should not be treated as a single CMV benchmark. Our project evaluates donor CMV classification and subset interpretation in the supplied cohort.

Approach	Representation or selection principle	Role here
CellCNN	Jointly learned cell filters, pooling and sample prediction	Implemented
Citrus	Hierarchical populations, sample-level features and regularised prediction	Original package used
Single-cell SVM	Cells inherit sample labels; a learned cell margin is aggregated	Linear SVM implemented; donor aggregation adapted
Single-cell logistic regression / random forest	Supervised prediction from individual cell profiles	Literature comparison only
Distance-based outlier detection	Distance to a nearest control-cell reference	Literature comparison only
Single-marker cutoff gates	Thresholds on individual markers select candidate cell subsets	Literature comparison only
Moment summary	First four moments of marker distributions represent each sample	Literature comparison only
Denoising autoencoder	Reconstruction of corrupted multi-cell inputs learns an unsupervised representation	Literature comparison only

Table S1: Approaches examined in the CellCNN publication, distinguished from the experiments performed for this project. We do not imply that every paper baseline was evaluated on this CMV cohort.

The original CellCNN implementation used Theano/Keras. Our implementation reproduces the relevant linear-filter/ReLU/pooling design in PyTorch, with explicitly recorded adaptations (S3). Citrus is the original R package [3]. Published results are background evidence; all numerical results in the main report come from this project’s current analyses.

S1.2 Analysis scopes and donor balance

The supplied archive derives from the study of NK-cell diversity by Horowitz et al. [2]; the supplied 20-donor subset is smaller than the publication’s complete cohort. The archive provides NK-gated cells and live-cell PBMC files for the same donors. Labels belong to donors and do not identify infected individual cells. Four complete twin pairs are identifiable from the supplied donor identities; related donors can therefore remain on opposite sides of a donor-wise split.

Equal cell counts reduce domination of pooled exploratory analyses by larger files. They do not make cells independent biological replicates. The NK-subsampling check compares an approximate NKG2C/CD57 phenotype between the balanced subset and the full NK gate (Fig. S1); it does not establish preservation of every rare population. The original inputs, labels and marker order are verified by file hashes. Finite negative marker values are retained, and no additional manual cell-type gate is fitted for classification.

Analysis	Gate	Sample and purpose
Dimensionality reduction	gated_NK	2,000 cells/donor; 40,000 final cells, 5,000 for parameter sweeps
Clustering	gated_alive	1,000 cells/donor; 20,000 cells clustered in 37D
Classification	gated_alive	Method-specific balanced training samples; donor-level test predictions
Subset maps	gated_alive	Fixed 500 cells/donor; 10,000 display cells, t-SNE perplexity 30
Subset marker profiles	gated_alive	Selected events from complete outer test donors
Bonus population evaluation	gated_alive	Fixed 20,000 evaluation cells/donor; a separate filter-specific frequency analysis

Table S2: The gates and sampling scopes are deliberately distinguished. The interpretation map is not the 40,000-cell NK embedding of the main dimensionality-reduction comparison.

Donor	CMV	Alive	NK	NK / alive (pct)	Test visits (30)
a__001	CMV+	82324	6320	7.6770	5
a__002	CMV+	108267	15809	14.6019	8
a__003	CMV-	97529	11438	11.7278	16
a__004	CMV-	140687	17443	12.3984	13
a__005	CMV+	155335	25226	16.2397	4
a__006	CMV-	90075	5652	6.2748	10
a__007	CMV+	104805	9819	9.3688	7
a__009	CMV-	216632	20401	9.4174	8
a__010	CMV-	122209	9092	7.4397	11
a__011	CMV-	258461	31424	12.1581	11
a__012	CMV+	187166	23528	12.5707	6
a__1a	CMV-	136515	7036	5.1540	8
a__2a	CMV-	236975	8739	3.6877	14
a__2b	CMV-	92403	3790	4.1016	10
a__3a	CMV-	180167	11605	6.4412	9
a__3b	CMV-	216648	13387	6.1791	10
a__4a	CMV+	311715	14933	4.7906	7
a__4b	CMV+	237274	5782	2.4368	7
a__5a	CMV+	254991	12104	4.7468	9
a__5b	CMV+	208572	8065	3.8668	7

Table S3: Donor inventory from the current raw-data summaries. Test visits are the appearances in the 30 common outer splits; the same counts apply to every method. NK/alive is a percentage, not an additional classifier feature.

Panel 1	Panel 2	Panel 3	Panel 4
CD3	CD10	CD11b	2DL3
CD27	3DL1-S1	NKp30	NKG2A
CD19	CD117	CD122	HLA-DR
CD4	2DS4	3DL1	2DL4
CD8	ILT2-CD85j	NKp44	CD56
CD57	NKp46	CD127	2DL5
2DL1-S1	NKG2D	2DL1	CD25
TRAIL	NKG2C	CD94	
2DL2-L3-S2	2B4	CD34	
CD16	CD33	CCR7	

Table S4: All 37 analysis markers. Columns are a typographical grouping, not separate experiments; analysis order follows the supplied marker file.

S1.3 Transformation and scaling

We apply $a_{ij} = \text{asinh}(x_{ij}/5)$ to each analysis marker. This compresses large intensities while retaining finite negative background measurements. The selected cofactor is fixed, not optimised by outer test performance. Exploratory maps and clustering use marker scaling fitted to their own balanced samples. CellCNN and SVM use a scaler fitted only on the relevant training donors. Citrus clusters unstandardised ArcSinh marker values; its logistic regression standardises cluster-frequency features internally.

The classification implementations preserve their original numerical paths: CellCNN/SVM convert raw values to float32 before ArcSinh, whereas Citrus uses the native float64 path. Marker means are accumulated in float64. For the new Citrus interpretation, all 127,233,750 raw marker values across 3,438,750 events were independently compared between FlowKit and native Citrus/FlowCore and agreed byte for byte.

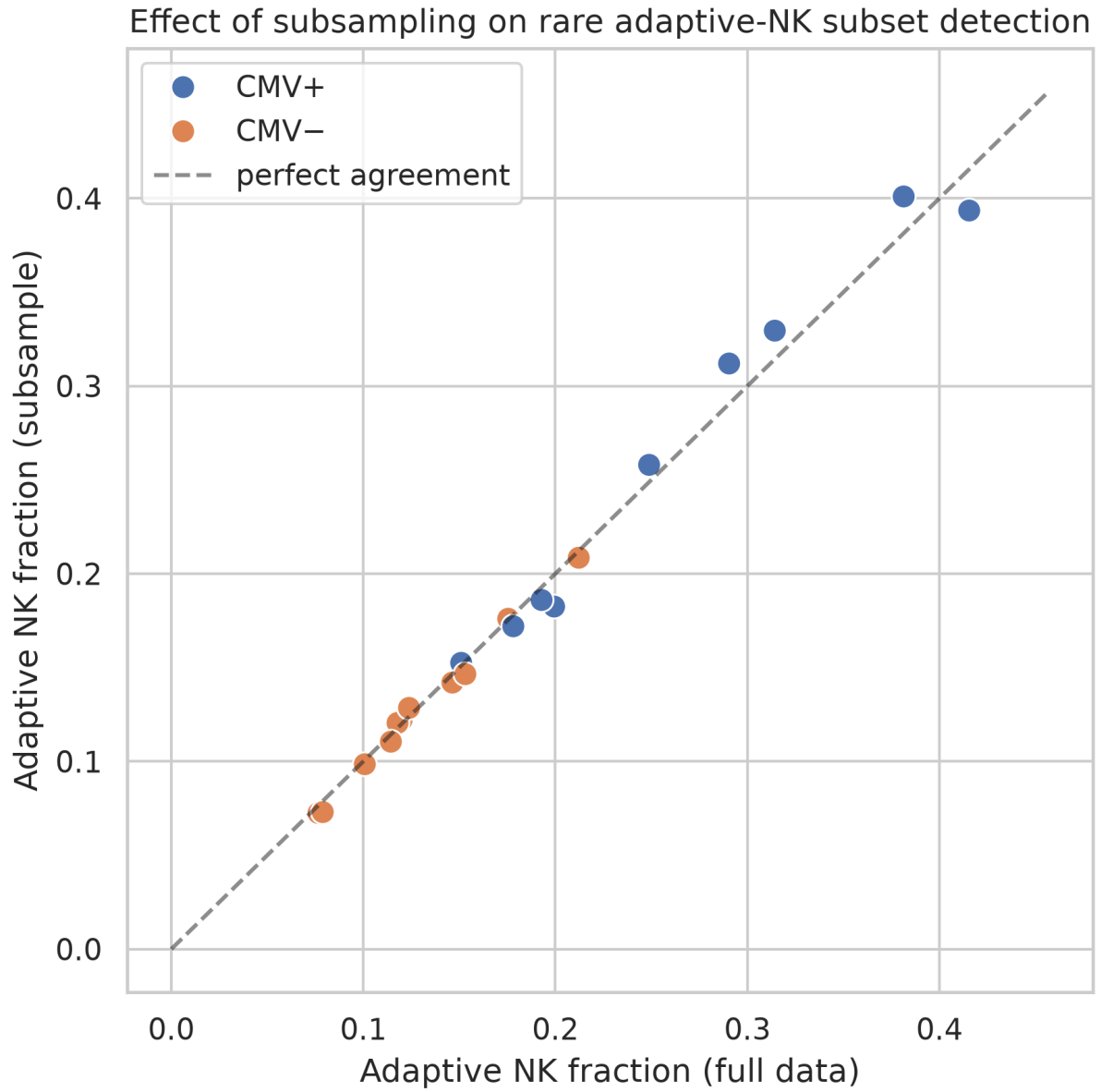


Figure S1: Phenotype-specific subsampling check within the NK gate. The “adaptive NK” axis label denotes an approximate NKG2C/CD57 phenotype defined by exploratory two-component mixture-model crossovers, not a validated functional cell type. Fractions refer to NK-gated cells, not all PBMCs; similarity here does not establish preservation of every rare population.

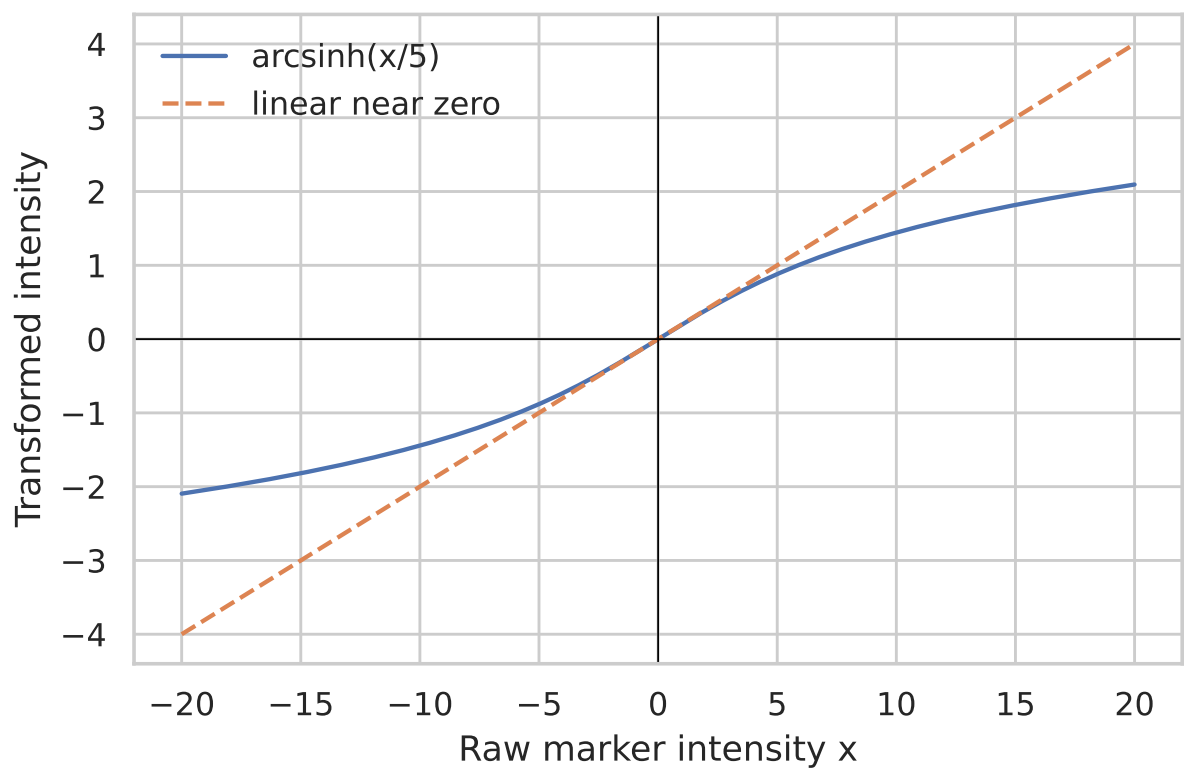


Figure S2: Analytical transformation $\text{asinh}(x/5)$ and its local linear approximation $x/5$. This curve illustrates compression of large intensities and retention of negative values; it is not an empirical marker distribution or a cofactor-optimisation result.

S2 Dimensionality reduction and clustering

S2.1 Projection parameters and comparison measures

PCA, t-SNE and UMAP use the same 40,000-cell NK sample. PCA fits 36 components; the first 29 retain 91.4978% of variance and define the high-dimensional reference for the nonlinear methods and quality scores. The first two retain 20.2911%. Retaining variance is not equivalent to proving removal of measurement noise.

t-SNE uses PCA initialisation, Barnes–Hut optimisation, 750 iterations and seed 42. Its six perplexities are 5, 30, 50, 60, 70 and 100. UMAP tests neighbours $\{5, 15, 50, 100\}$, minimum distance $\{0, 0.1, 0.5, 1\}$ and learning rate $\{0.1, 0.5, 1, 5\}$, with seed 42 and otherwise the recorded library defaults. The sweep uses a fixed 5,000-cell subset of the selected PC scores. Additional tunable choices include PCA whitening/solver, t-SNE learning rate/initialisation and the metric, initialisation and iteration settings of UMAP; they are not additional experimental factors in this analysis.

For neighbourhood size $k = 15$, trustworthiness is

$$T(k) = 1 - \frac{2}{nk(2n - 3k - 1)} \sum_i \sum_{j \in U_k(i)} (r(i, j) - k),$$

where $U_k(i)$ contains low-dimensional neighbours absent from the high-dimensional neighbourhood and $r(i, j)$ is their high-dimensional rank [6]. False neighbours are penalised more when originally distant. The final 40,000-cell calculation uses exact ranks in row blocks. kNN preservation is the mean fraction of shared neighbours, $|N_k^A(i) \cap N_k^B(i)|/k$; it gives no partial credit to near misses. Distance correlation is Pearson correlation of pairwise distances within one fixed 2,000-cell subset.

For Procrustes comparison, corresponding cells are aligned by centring, rotation/reflection and scaling; the sum of squared residuals is the reported disparity M^2 [9]. It evaluates aligned global shape, whereas neighbourhood overlap evaluates local agreement. Both are geometric comparisons, not biological validation.

Perplexity	Trustworthiness	kNN overlap
5	0.8931	0.2527
30	0.9235	0.2835
50	0.9264	0.2831
60	0.9244	0.2794
70	0.9271	0.2774
100	0.9246	0.2677

Table S5: The complete t-SNE sweep. Perplexity 70 wins by trustworthiness; perplexity 30 has slightly higher exact-neighbour overlap. Parameter selection therefore depends on the declared criterion.

Learning rate	Mean trustworthiness (two fixed settings)
0.1000	0.8781
0.5000	0.8767
1.0000	0.8723
5.0000	0.8589

Table S6: UMAP learning-rate selection: mean trustworthiness of the two fixed configurations $(n, d) = (5, 0)$ and $(50, 0.1)$. Learning rate 0.1 wins this mean; the best single configuration across all rates is not the selection rule.

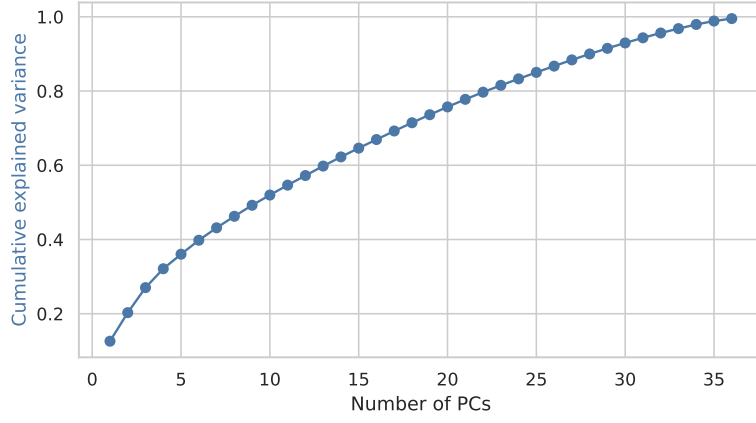


Figure S3: PCA spectrum and the retained 29-component representation. The two-dimensional PCA display contains considerably less variance than the nonlinear methods' 29-dimensional input.

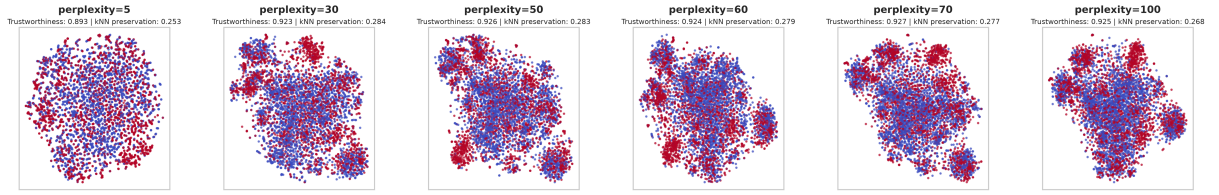


Figure S4: Saved t-SNE parameter examples on the sweep subset. Perplexity changes the neighbourhood scale and visual organisation; these layouts should not be interpreted as independently annotated populations.

Embedding	Trustworthiness	kNN overlap	Distance r
PCA	0.7258	0.0072	0.6204
t-SNE	0.9608	0.2162	0.4799
UMAP	0.8838	0.1122	0.4317

Table S7: Final embedding quality against the 29-PC reference. Trustworthiness and kNN overlap use 40,000 cells; distance correlation uses the fixed 2,000-cell subset.

Pair	kNN overlap	Procrustes
PCA vs t-SNE	0.0060	0.4422
PCA vs UMAP	0.0043	0.5542
t-SNE vs UMAP	0.1277	0.3683

Table S8: All three pairwise comparisons of the final 40,000-cell embeddings. t-SNE/UMAP has both the highest neighbour overlap and lowest Procrustes disparity.

Table S9: All 64 UMAP configurations on the fixed 5,000-cell sweep sample.

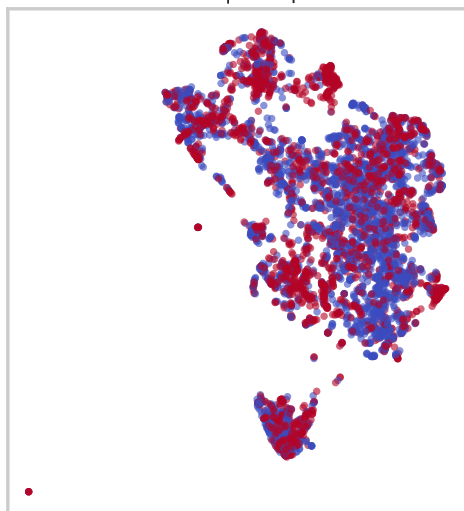
Learning rate	Neighbours	Min. distance	Trustworthiness	kNN overlap
0.1000	5	0.0000	0.8902	0.2236
0.1000	5	0.1000	0.8732	0.2053
0.1000	5	0.5000	0.8180	0.1222
0.1000	5	1.0000	0.7600	0.0729
0.1000	15	0.0000	0.8888	0.1994
0.1000	15	0.1000	0.8703	0.1715
0.1000	15	0.5000	0.8255	0.1068
0.1000	15	1.0000	0.7975	0.0794
0.1000	50	0.0000	0.8807	0.1694
0.1000	50	0.1000	0.8660	0.1463
0.1000	50	0.5000	0.8279	0.0958
0.1000	50	1.0000	0.8066	0.0748
0.1000	100	0.0000	0.8736	0.1549
0.1000	100	0.1000	0.8572	0.1330
0.1000	100	0.5000	0.8271	0.0930
0.1000	100	1.0000	0.8082	0.0740
0.5000	5	0.0000	0.8916	0.2184
0.5000	5	0.1000	0.8829	0.1976
0.5000	5	0.5000	0.8183	0.1166
0.5000	5	1.0000	0.7884	0.0813
0.5000	15	0.0000	0.8842	0.1891
0.5000	15	0.1000	0.8674	0.1645
0.5000	15	0.5000	0.8219	0.1018
0.5000	15	1.0000	0.8043	0.0773
0.5000	50	0.0000	0.8746	0.1587
0.5000	50	0.1000	0.8618	0.1397
0.5000	50	0.5000	0.8246	0.0939
0.5000	50	1.0000	0.8078	0.0759
0.5000	100	0.0000	0.8662	0.1449
0.5000	100	0.1000	0.8545	0.1280
0.5000	100	0.5000	0.8230	0.0902
0.5000	100	1.0000	0.8049	0.0732
1.0000	5	0.0000	0.8882	0.2142
1.0000	5	0.1000	0.8727	0.1950
1.0000	5	0.5000	0.8213	0.1170
1.0000	5	1.0000	0.7908	0.0807
1.0000	15	0.0000	0.8848	0.1855
1.0000	15	0.1000	0.8651	0.1604
1.0000	15	0.5000	0.8236	0.1022
1.0000	15	1.0000	0.8042	0.0764
1.0000	50	0.0000	0.8712	0.1528
1.0000	50	0.1000	0.8565	0.1344
1.0000	50	0.5000	0.8274	0.0939
1.0000	50	1.0000	0.8059	0.0743
1.0000	100	0.0000	0.8601	0.1366
1.0000	100	0.1000	0.8522	0.1235
1.0000	100	0.5000	0.8227	0.0868
1.0000	100	1.0000	0.8053	0.0722
5.0000	5	0.0000	0.8741	0.1939
5.0000	5	0.1000	0.8600	0.1766
5.0000	5	0.5000	0.8183	0.1145
5.0000	5	1.0000	0.7939	0.0795
5.0000	15	0.0000	0.8613	0.1556
5.0000	15	0.1000	0.8498	0.1391
5.0000	15	0.5000	0.8248	0.0997

Continued on next page

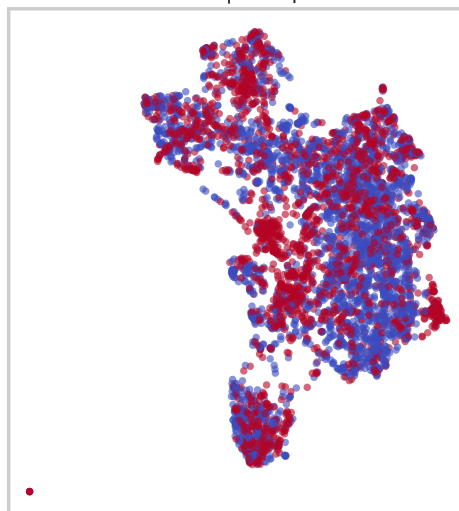
Table S9: All 64 UMAP configurations on the fixed 5,000-cell sweep sample.

Learning rate	Neighbours	Min. distance	Trustworthiness	kNN overlap
5.0000	15	1.0000	0.8009	0.0748
5.0000	50	0.0000	0.8514	0.1235
5.0000	50	0.1000	0.8437	0.1128
5.0000	50	0.5000	0.8228	0.0883
5.0000	50	1.0000	0.8036	0.0710
5.0000	100	0.0000	0.8424	0.1051
5.0000	100	0.1000	0.8386	0.1004
5.0000	100	0.5000	0.8125	0.0774
5.0000	100	1.0000	0.7980	0.0647

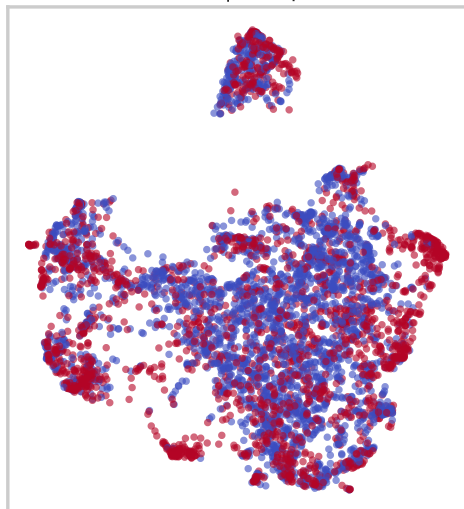
nn=5, md=0.0, lr=0.1
Trustworthiness: 0.890 | kNN preservation: 0.224



nn=5, md=0.1, lr=0.1
Trustworthiness: 0.873 | kNN preservation: 0.205



nn=15, md=0.0, lr=0.1
Trustworthiness: 0.889 | kNN preservation: 0.199



nn=15, md=0.1, lr=0.1
Trustworthiness: 0.870 | kNN preservation: 0.172

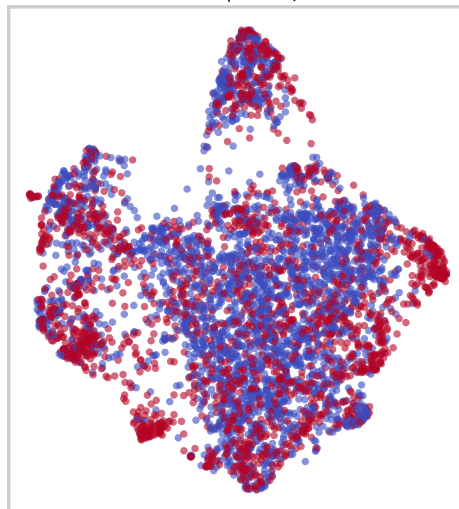
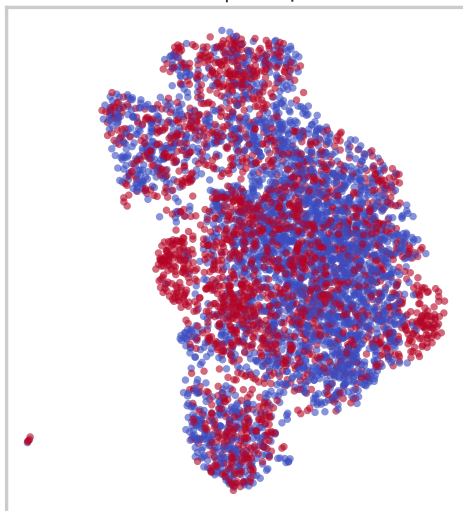
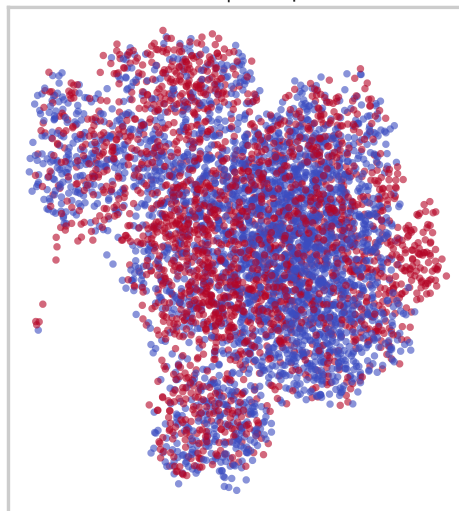


Figure S5: UMAP parameter examples at learning rate 0.1, first part, left half of the saved grid. Splitting the original vector figure preserves readable parameter labels; all numerical results are in Table S9.

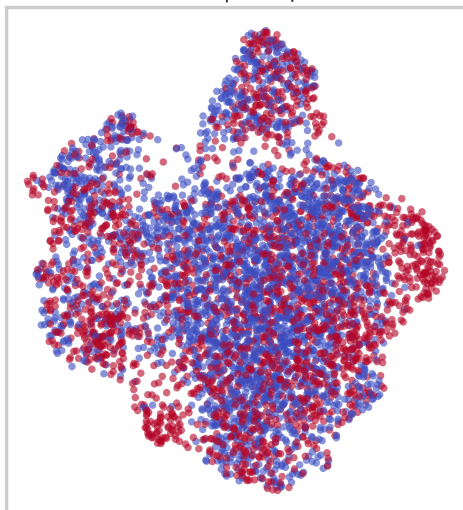
nn=5, md=0.5, lr=0.1
Trustworthiness: 0.818 | kNN preservation: 0.122



nn=5, md=1, lr=0.1
Trustworthiness: 0.760 | kNN preservation: 0.073



nn=15, md=0.5, lr=0.1
Trustworthiness: 0.825 | kNN preservation: 0.107



nn=15, md=1, lr=0.1
Trustworthiness: 0.797 | kNN preservation: 0.079

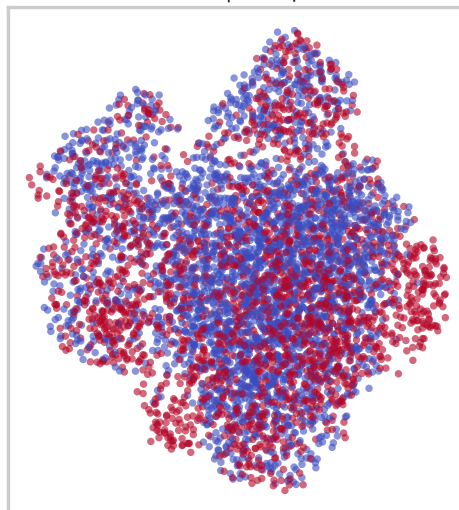
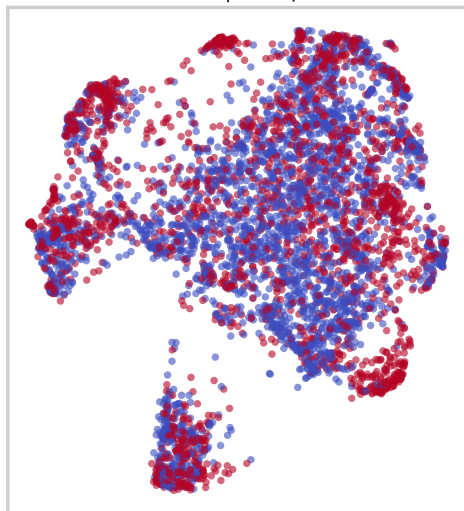
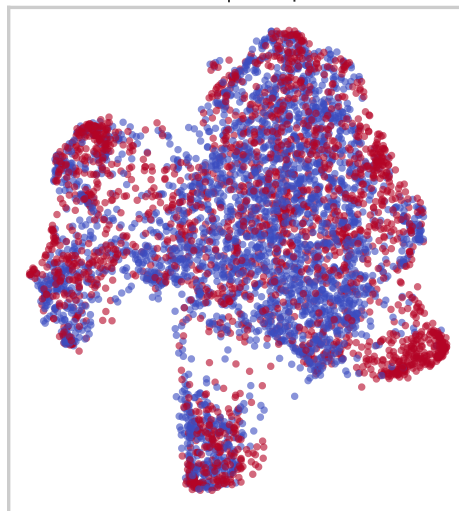


Figure S6: UMAP parameter examples at learning rate 0.1, first part, right half.

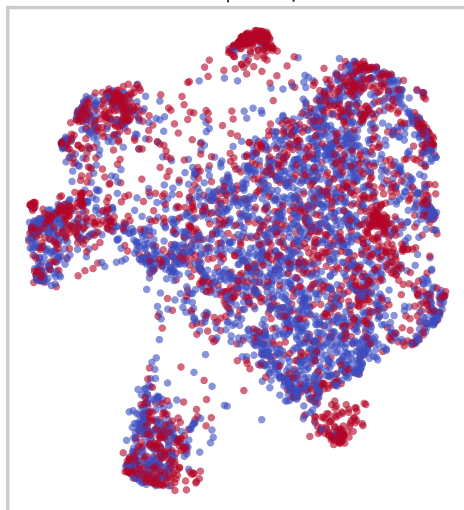
nn=50, md=0.0, lr=0.1
Trustworthiness: 0.881 | kNN preservation: 0.169



nn=50, md=0.1, lr=0.1
Trustworthiness: 0.866 | kNN preservation: 0.146



nn=100, md=0.0, lr=0.1
Trustworthiness: 0.874 | kNN preservation: 0.155



nn=100, md=0.1, lr=0.1
Trustworthiness: 0.857 | kNN preservation: 0.133

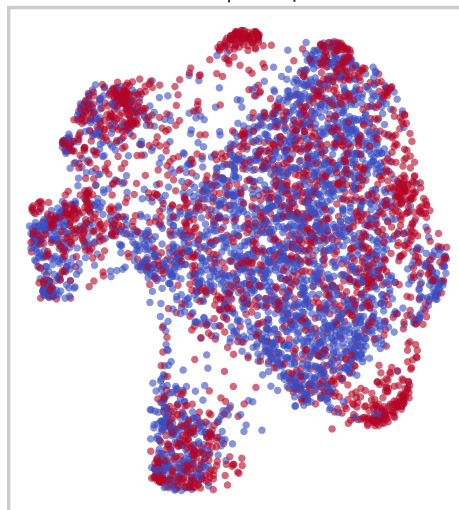
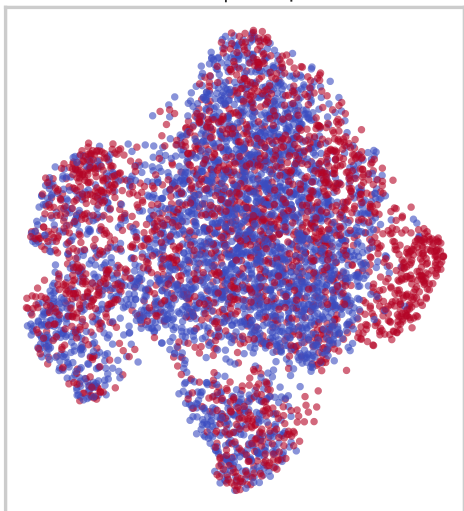
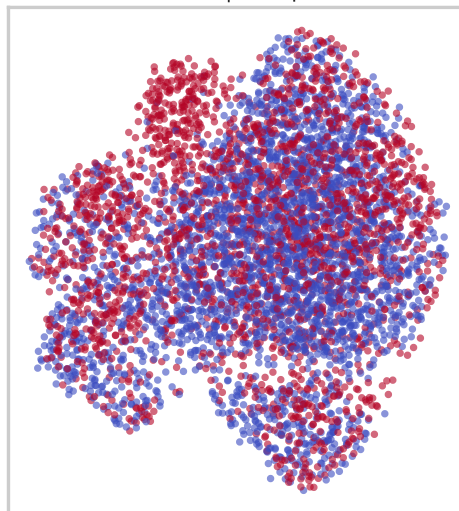


Figure S7: UMAP parameter examples at learning rate 0.1, second part, left half.

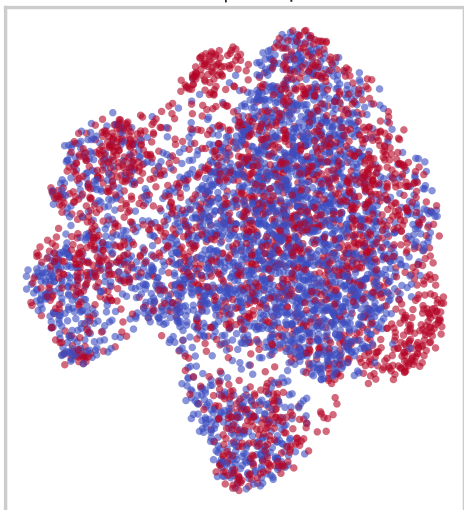
nn=50, md=0.5, lr=0.1
Trustworthiness: 0.828 | kNN preservation: 0.096



nn=50, md=1, lr=0.1
Trustworthiness: 0.807 | kNN preservation: 0.075



nn=100, md=0.5, lr=0.1
Trustworthiness: 0.827 | kNN preservation: 0.093



nn=100, md=1, lr=0.1
Trustworthiness: 0.808 | kNN preservation: 0.074

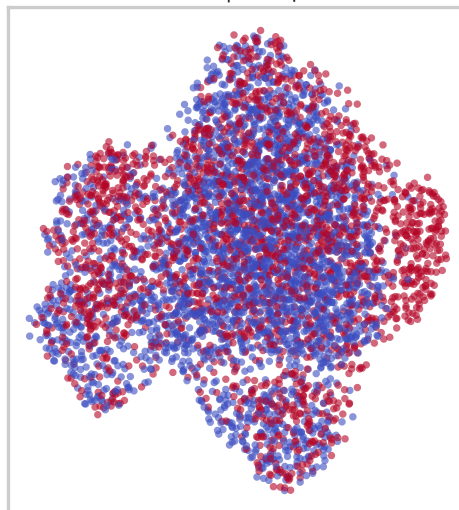


Figure S8: UMAP parameter examples at learning rate 0.1, second part, right half.

S2.2 Clustering grid and stability

The clustering sample is drawn by one NumPy generator with seed 42, advanced sequentially in the supplied donor-file order. Each donor contributes 1,000 finite-marker events. K-means uses ten initialisations for $k \in \{2, 3, 4, 6, 8, 10, 12\}$. Hierarchical clustering evaluates the same cuts for single, average, complete and Ward linkage. Leiden uses a weighted undirected neighbour graph constructed with Scanpy, neighbour counts $\{10, 30, 60\}$, resolutions $\{0.3, 0.7, 1.2\}$, the `leidenalg` backend and convergence iterations [8]. All fits and DB scores use standardised 37D data.

DB gives each cluster equal weight:

$$DB = \frac{1}{K} \sum_c \max_{j \neq c} \frac{S_c + S_j}{\|\mu_c - \mu_j\|},$$

where S_c is mean distance to centroid μ_c [7]. This differs from a cell-weighted mean silhouette, but does not protect against small artefactual groups. Single, average and complete linkage select partitions dominated by one giant cluster; all are retained in the complete results rather than silently removed.

For each of ten repeats, 80% of the initially sampled cells are retained within each donor. Scaling and clustering are refitted. On retained cells, each original cluster takes its maximum Jaccard match to a new cluster. We first take a median across assessed repeats for each cluster, then an unweighted mean over clusters. An absent cluster is unassessable in that repeat; a singleton may score one whenever retained. The stability cutoff 0.6 is heuristic. This procedure tests cell removal and algorithmic/scaling changes within the original sample, not generalisation to new donors.

Method	Parameters	Clusters	DB ↓	Stability	Largest cluster
K-means	$k = 4$	4	2.363	0.995	37.780 %
Single	$k = 2$	2	0.264	1.000	99.995 %
Average	$k = 2$	2	0.894	0.602	99.975 %
Complete	$k = 2$	2	0.948	0.617	99.960 %
Ward	$k = 6$	6	2.522	0.611	38.220 %
Leiden	$n = 30, r = 1.2$	18	2.361	0.683	17.940 %

Table S10: Selected configurations for every method/linkage, including the degenerate hierarchical solutions. A high largest-cluster fraction reveals information not captured by the score alone.

Table S11: The original 44 clustering configurations. Scores and stability are stored at three-decimal precision.

ID	Method	k	Neighbours	Resolution	Clusters	DB	Stability
0	k-means	2	—	—	2	2.4210	0.9970
1	k-means	3	—	—	3	2.4220	0.9960
2	k-means	4	—	—	4	2.3630	0.9950
3	k-means	6	—	—	6	2.4280	0.9880
4	k-means	8	—	—	8	2.5630	0.9750
5	k-means	10	—	—	10	2.6480	0.9730
6	k-means	12	—	—	12	2.6080	0.7690
7	Single	2	—	—	2	0.2640	1.0000
8	Single	3	—	—	3	0.3020	0.6670
9	Single	4	—	—	4	0.3030	1.0000
10	Single	6	—	—	6	0.4250	1.0000
11	Single	8	—	—	8	0.4120	0.8750
12	Single	10	—	—	10	0.3290	1.0000
13	Single	12	—	—	12	0.3340	1.0000
14	Average	2	—	—	2	0.8940	0.6020
15	Average	3	—	—	3	1.6300	0.4490
16	Average	4	—	—	4	1.5840	0.4270
17	Average	6	—	—	6	1.6040	0.5020
18	Average	8	—	—	8	1.6930	0.4740
19	Average	10	—	—	10	1.4790	0.6170

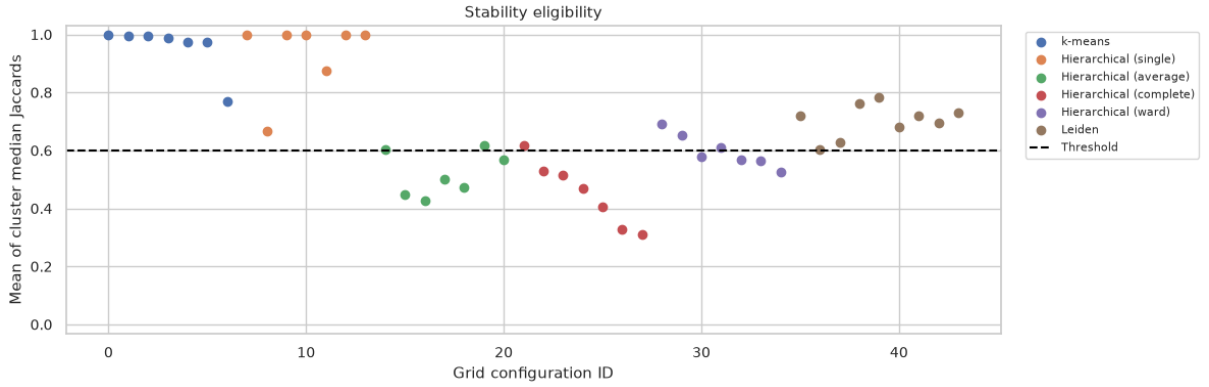
Continued on next page

Table S11: The original 44 clustering configurations. Scores and stability are stored at three-decimal precision.

ID	Method	k	Neighbours	Resolution	Clusters	DB	Stability
20	Average	12	–	–	12	1.6390	0.5680
21	Complete	2	–	–	2	0.9480	0.6170
22	Complete	3	–	–	3	1.5150	0.5290
23	Complete	4	–	–	4	1.7090	0.5140
24	Complete	6	–	–	6	2.1150	0.4700
25	Complete	8	–	–	8	2.7230	0.4050
26	Complete	10	–	–	10	2.9760	0.3290
27	Complete	12	–	–	12	2.8250	0.3090
28	Ward	2	–	–	2	2.5530	0.6910
29	Ward	3	–	–	3	2.8850	0.6530
30	Ward	4	–	–	4	2.7700	0.5790
31	Ward	6	–	–	6	2.5220	0.6110
32	Ward	8	–	–	8	2.9800	0.5670
33	Ward	10	–	–	10	2.7930	0.5630
34	Ward	12	–	–	12	2.8300	0.5270
35	Leiden	–	10	0.3	7	2.3920	0.7190
36	Leiden	–	10	0.7	13	2.5570	0.6040
37	Leiden	–	10	1.2	21	2.4960	0.6270
38	Leiden	–	30	0.3	6	2.4430	0.7620
39	Leiden	–	30	0.7	9	2.5230	0.7850
40	Leiden	–	30	1.2	18	2.3610	0.6830
41	Leiden	–	60	0.3	5	2.5810	0.7190
42	Leiden	–	60	0.7	10	2.4790	0.6940
43	Leiden	–	60	1.2	17	2.4330	0.7310

Table S12: Cluster-wise stability for the three illustrated partitions.

Method	Cluster	Cells	Median J	Q1	Q3	Repeats
k-means	0	4301	0.9940	0.9910	0.9940	10
k-means	1	4995	0.9940	0.9930	0.9950	10
k-means	2	7556	0.9970	0.9960	0.9970	10
k-means	3	3148	0.9950	0.9940	0.9950	10
Ward	0	2872	0.5730	0.5450	0.5910	10
Ward	1	1911	0.5830	0.5230	0.6390	10
Ward	2	6367	0.6880	0.6410	0.7000	10
Ward	3	7644	0.7060	0.6670	0.7350	10
Ward	4	519	0.4820	0.1740	0.7340	10
Ward	5	687	0.6350	0.6060	0.6600	10
Leiden	0	3588	0.7800	0.6770	0.8950	10
Leiden	1	2881	0.5490	0.4830	0.9180	10
Leiden	2	2200	0.6510	0.6450	0.7860	10
Leiden	3	2035	0.8440	0.8360	0.9140	10
Leiden	4	1887	0.7630	0.7540	0.7820	10
Leiden	5	1165	0.5680	0.5520	0.6940	10
Leiden	6	1072	0.6970	0.6540	0.7160	10
Leiden	7	857	0.8450	0.7590	0.8500	10
Leiden	8	757	0.7460	0.7310	0.8190	10
Leiden	9	615	0.9160	0.9020	0.9230	10
Leiden	10	531	0.8360	0.8200	0.8500	10
Leiden	11	506	0.8480	0.8100	0.8610	10
Leiden	12	420	0.8540	0.8310	0.8990	10
Leiden	13	401	0.7600	0.7470	0.7680	10
Leiden	14	297	0.6670	0.1370	0.8090	10
Leiden	15	284	0.1110	0.1050	0.3190	10
Leiden	16	281	0.1200	0.1170	0.1220	10
Leiden	17	223	0.7340	0.5820	0.7640	10

**Figure S9:** Original stability eligibility results for the full grid. High average stability can coexist with weak individual clusters.

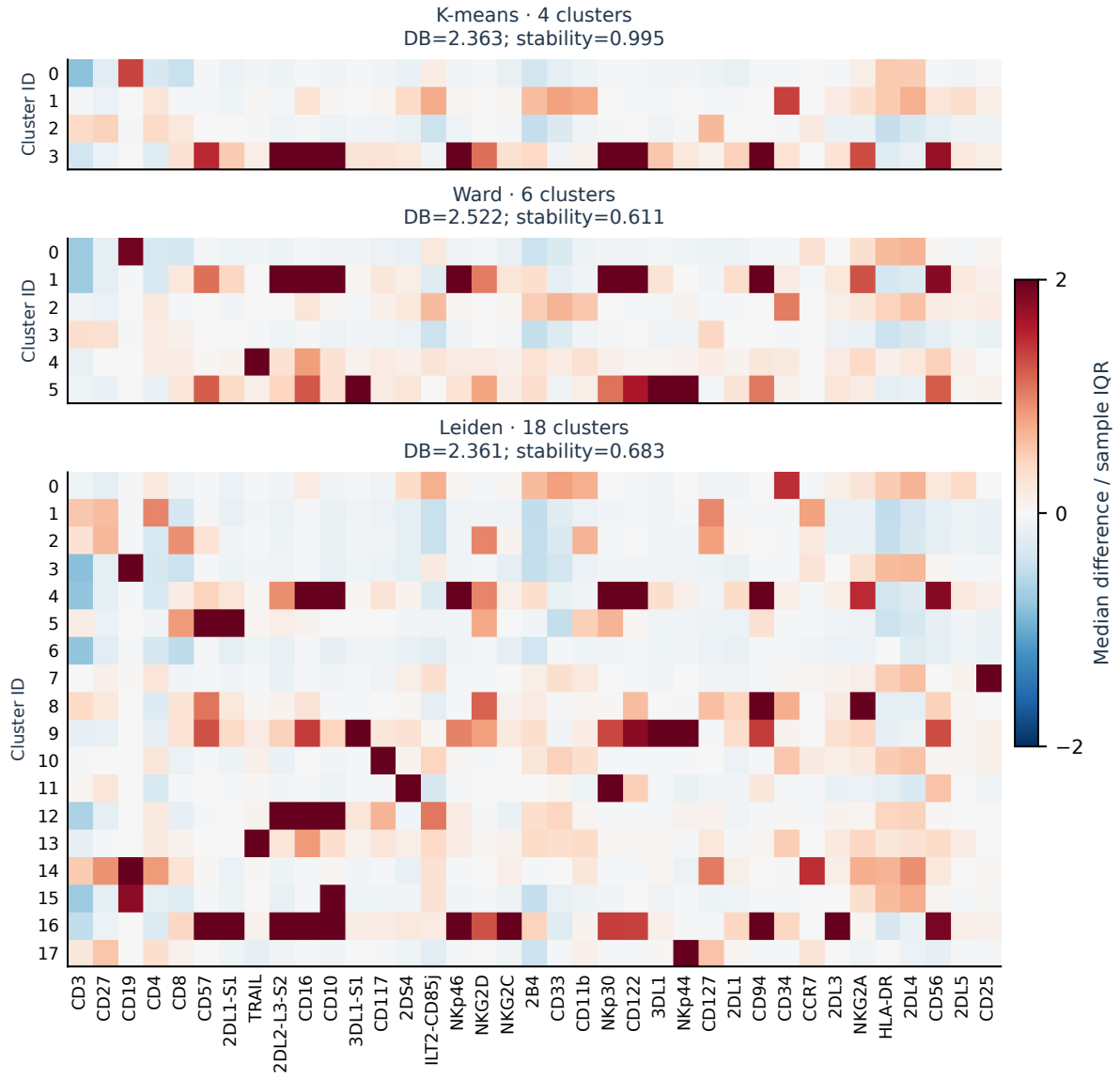


Figure S10: All 37 markers for the three selected partitions. Relative expression is cluster median minus sample median, divided by sample IQR, after ArcSinh. Display colours are clipped to ± 2 ; unclipped numerical profiles are exported in `data/cluster_profiles.csv`. Marker averages do not establish co-expression within individual cells.

S3 Classification and subset implementation

S3.1 Shared splits and model selection

Splits 0–29 are the common three-method benchmark; CellCNN/SVM also have 100 stored splits. Every outer training set has 14 donors (7 CMV-positive, 7 negative), and each test set has two positive and four negative donors. Three fixed inner folds are shared across methods. These are repeated holdouts of the same 20 people, not 30 independent studies. All scaler fits and training inputs exclude outer test donors.

Method	Training and representation	Inner selection and outer evaluation
CellCNN	200 bags of 3,000 cells per inner training donor; 20,000 cells/donor for scaler fitting; 3, 4 or 5 linear/ReLU filters; top-1% mean pooling; two-output layer	Nine candidates (three folds $\times$ three filter counts). Validation accuracy, then AUC, regularised loss, fewer filters and fold order. Retain the winning network; average five test-input probabilities
Citrus	10,000 cells/training donor; Ward tree; clusters from 0.05% of training cells; abundance features; L1-logistic regression	Original <code>cv.min</code> inner-error selection. Final tree and regression use all 14 training donors; 10,000 sampled cells/test donor for benchmark predictions
Linear SVM	10,000 cells/training donor; donor labels assigned to cells; training-only scaler; L2 squared-hinge loss	$C \in \{0.01, 0.1, 1\}$ by mean inner AUC, smaller C for ties; refit on all 14 training donors; full-donor top-1% mean margin

Table S13: Implementation details that materially determine the comparison. CellCNN is a paper-guided reimplementaion; Citrus uses the original package; SVM donor aggregation is a project adaptation.

CellCNN uses Adam with learning rate 0.01, batch size 128, L2 coefficient 10^{-4} , at most 100 epochs and patience five. The validation-loss improvement tolerance is 10^{-6} . No dropout is used. Inner training and validation bags are materialised with separate seeds. The retained network was trained on nine or ten inner training donors; it is not refitted on all 14. Validation and test average five probabilities from up to 20,000 sampled cells per input. A fixed 3/4/5-filter search replaces the publication’s random search; averaging multiple test inputs is another adaptation.

For Citrus, final training uses 140,000 cells. Its original mapping attaches a new cell to the nearest training event, then transfers that event’s memberships in the nested hierarchy. Regression operates on cluster frequencies, not cell labels. The minimum cluster size is 70 events in the final training set. The existing lambda path is constructed using the full outer training set, while inner trees and fitted weights use the specified training folds. Thirty splits and 10,000 cells/donor are computational reductions relative to the larger paper protocol.

For SVM, $m_i = w^T z_i + b$ is a margin, not a probability. With I_d the highest-scoring $k_d = \lceil 0.01N_d \rceil$ cells, $s_d = k_d^{-1} \sum_{i \in I_d} m_i$. The threshold maximises sensitivity plus specificity on inner out-of-fold donor scores. CellCNN/Citrus use threshold 0.5; SVM retains its inner-trained threshold. AUC and AP use scores, whereas BA uses the declared thresholded decisions. AP summarises precision at positive-recall increments; it is not an alternative name for trapezoidal PR area.

S3.2 Quantitative cell-subset scores

For cell i , split s , filter f and cluster j , the positive-direction scores are

$$\begin{aligned}
 q_{is}^{\text{CNN}} &= \max_{f: \Delta_{fs} > 0, M_{fs} > 0} r_{ifs} / M_{fs}, & q_{is}^{\text{CNN}} &> 0.5, \\
 q_{is}^{\text{SVM}} &= m_{is} - \tau_s \quad (i \in I_{ds}), & q_{is}^{\text{SVM}} &> 0, \\
 q_{is}^{\text{Citrus}} &= \max(\{0\} \cup \{\beta_{js} : i \in A_{js}\}), & q_{is}^{\text{Citrus}} &> 10^{-10}.
 \end{aligned}$$

Here $\Delta_{fs} = v_{f,+} - v_{f,-}$ is the output-weight contrast. CellCNN’s M_{fs} is the maximum response on the original scaler-reference draw: 20,000 cells per actual inner training donor with the original

candidate seed. It is never recomputed from test or display cells. A filter with zero reference maximum is ineligible; an empty maximum is defined as zero. New cells may exceed the reference maximum. SVM cells outside I_{ds} are unselected; boundary ties use the original FCS event index. Citrus’s A_{js} denotes transferred training-cluster membership, and 10^{-10} excludes numerical zeros.

The plotted cell frequency is

$$f_i = \frac{\sum_{s \in \mathcal{T}_d} \mathbf{1}\{\text{cell } i \text{ passes the method's selection rule in } s\}}{|\mathcal{T}_d|},$$

where $\mathcal{T}_d$ contains all outer test appearances of its donor. Each cell counts at most once per split even if several filters or nested clusters select it. All methods use identical denominators of 4–16 visits. In Citrus, splits 19, 24 and 26 contain no effective clusters; split 27 contains only negative coefficients. All four therefore contribute zero positive selections and remain in the denominator.

The display reference is the original 10,000-cell live-PBMC t-SNE map at perplexity 30, with its stored exploratory scaler. It is distinct from the NK-gated Task-2 map. Marker profiles use selected *full-donor* cells: mean cells per nonempty visit, mean visits per donor, then equal means over contributing donors. A visit without selected cells has no marker profile, rather than a vector of zeros. Both CMV classes contribute; all methods have 20 contributing donors. The common display transformation is $(c_j - \mu_j)/\sigma_j$, with μ_j, σ_j from the reference map, not model weights.

S3.3 Reconstruction and numerical checks

The original Citrus exports retained coefficients and cluster means but not tree memberships. We reconstructed only the original final Ward trees with their original donor draws and seeds, retaining the selected classifiers. Every saved cluster profile in a reconstructed tree agrees within 5.33×10^{-15} . Full test-donor cells are then mapped to these trees; original 10,000-cell benchmark predictions are not replaced by new full-donor predictions.

Exact nearest-training-cell selection is accelerated by proving nonselection when an actual point outside the positive cluster union is closer than the exact nearest positive member. An approximate query supplies only possible negative witnesses; unresolved cases use an exact full search, with explicit first-row tie handling. “Negative” here means outside the selected training union, not a CMV-negative donor. Across 26 models with positive selections, 9,984 native Citrus control queries match nearest indices and decisions exactly. Caches bind data, code, tree files and numerical package versions. Separate numerical checks also recomputed donor metrics, aggregation and additional exhaustive nearest-neighbour cases. These checks establish implementation consistency, not biological validity.

Component	Recorded implementation
Python analysis	NumPy 2.5.2; pandas 2.3.3; SciPy 1.18.0
Classical learning / projections	scikit-learn 1.9.0; Scanpy 1.12.4; umap-learn 0.5.12
Neural models / FCS reader	PyTorch 2.14.0; FlowKit 1.3.2
Citrus runtime	R 4.5.3; Citrus 0.8; Rclusterpp 0.2.6

Table S14: Recorded software context. Native Citrus and Rclusterpp commits and full input/code hashes are retained in the repository provenance files.

S4 Modified CellCNN: detailed methods

S4.1 Common protocol and linear baseline

All four models use the same 100 outer donor splits and the same three-fold/filter-count candidate design. The baseline is the original linear/ReLU CellCNN. Training bag size, sampling, Adam settings, filter counts, early stopping and five-input test averaging remain as in S3. For the baseline and hard quadratic model, the per-filter pooled response is the mean of the largest $\max(1, \lfloor 0.01N \rfloor)$ responses. This floor convention differs from SVM’s ceiling convention; both are retained faithfully.

S4.2 Diagonal quadratic responses

For standardised cell vector z , filter f computes

$$r_f(z) = \max\left(0, b_f + w_f^T z + q_f^T z^{\odot 2}\right).$$

There are no cross-marker products $z_j z_\ell$ for $j \neq \ell$. Positive or negative squared-marker coefficients allow non-monotone responses in individual markers. The new quadratic weights start at zero; the hard-pooling model therefore starts with the same linear response and random initialisation sequence as the baseline. Its loss is cross-entropy plus 10^{-4} times the squared linear, quadratic and output weight matrices; biases are not included in that penalty.

S4.3 Learnable soft-fraction pooling

Sort responses within a bag in descending order, $r_{(j)f}$, and assign midpoint ranks $u_j = (j - \frac{1}{2})/N$, $j = 1, \dots, N$. The learnt fraction and pooling weights are

$$\alpha_f = \text{sigmoid}(\theta_f), \quad g_{jf} = \text{sigmoid}\left(\frac{\alpha_f - u_j}{0.002}\right), \quad h_f = \frac{\sum_j g_{jf} r_{(j)f}}{\max(\sum_j g_{jf}, 10^{-6})}.$$

Each filter starts at $\alpha_f = 0.01$. The fraction is bounded in $(0, 1)$ and trained jointly with the classifier. This is a smooth weighting of sorted responses, not a binary selection mask, and is not exactly hard top-1% at its initial finite temperature. The quadratic soft model uses the same diagonal response and weight penalties as the hard quadratic model.

S4.4 Mahalanobis/ReLU prototype variant

The prototype response is

$$d_f(z) = \max\left(0, \frac{1}{37} \sum_{j=1}^{37} a_{fj} (z_j - c_{fj})^2\right), \quad r_f(z) = \max(0, \rho_f - d_f(z)).$$

Positive weights are parameterised as $\tilde{a}_{fj} = \text{softplus}(u_{fj}) + 10^{-6}$ and normalised to mean one within each filter. Radii use $\rho_f = \text{softplus}(v_f) + 10^{-6}$. Thus the metric is diagonal; a full covariance matrix is not learned. Soft pooling is defined as above.

Initial centres are sampled from a donor-balanced training-only pool of 2,000 cells per donor, using a separate seed offset of 300,000. Weights start equally, and each initial radius is the 1% quantile of its prototype distances, bounded below by 2×10^{-6} . Training adds output-weight L2 (10^{-4}) and a geometry penalty $10^{-3} \text{mean}[(a - 1)^2]$; centres, radii and geometry are not given the quadratic model’s weight penalty. Validation loss contains cross-entropy and output L2 but excludes the geometry penalty. Consequently this is an exploratory architecture comparison, not an isolated test of response geometry with every regulariser held identical.

S4.5 Separate population-frequency evaluation

Bonus frequency AUC uses one filter per model and split: the filter with the largest positive output contrast. Its half-maximum threshold comes from the actual inner-training reference. Evaluation uses a fixed 20,000-cell sample per donor (seed 63,000 plus sorted donor index). This is distinct from the union-of-positive-filters definition used in Task 5.

The baseline lacks a positive filter in splits 44 and 84. Those frequencies remain missing, not zero. Population-frequency comparisons use the same 98 jointly evaluable splits for all four models, whereas network-score comparisons use all 100. Frequency effect means positive-donor frequency minus negative-donor frequency within a split; it is not a causal disease effect.

S5 Additional results

S5.1 Classification distributions

Method	AUC		AP		BA	
CellCNN	0.875	[0.625, 1.000]	0.833	[0.700, 1.000]	0.625	[0.500, 0.875]
SVM	0.875	[0.750, 0.875]	0.833	[0.712, 0.833]	0.625	[0.500, 0.750]
Citrus	0.625	[0.500, 0.750]	0.542	[0.342, 0.750]	0.500	[0.500, 0.625]

Table S15: Three-method benchmark on the 30 common splits: median [Q1, Q3]. AUC and AP are score-based; BA uses the method’s declared decision threshold.

Method	AUC	AP	BA
CellCNN	0.8250	0.8400	0.6792
SVM	0.8083	0.7950	0.6250
Citrus	0.6062	0.5783	0.5292

Table S16: Means over the same 30 splits. Means and medians are deliberately reported separately.

Method (100 splits)	AUC		AP		BA	
CellCNN	0.875	[0.750, 1.000]	0.833	[0.700, 1.000]	0.750	[0.500, 0.781]
SVM	0.875	[0.625, 1.000]	0.833	[0.692, 1.000]	0.625	[0.500, 0.750]

Table S17: Additional CellCNN/SVM comparison over all 100 stored splits: median [Q1, Q3]. This is a separate comparison, not an extension of Citrus to 100 splits.

Comparison	Mean	Median	Q1	Q3	Better	Equal	Worse
CellCNN - SVM	0.0167	0.0000	0.0000	0.1250	12	12	6
CellCNN - Citrus	0.2188	0.1875	0.0156	0.3750	22	6	2
SVM - Citrus	0.2021	0.1250	0.0000	0.3750	19	6	5

Table S18: Paired AUC differences on identical donor splits. Better/equal/worse counts describe the first method relative to the second. No independence-based significance test is inferred from these overlapping repeats.

S5.2 Subset selection and recurrence

Method	Any visit	At least half	Every visit	Nonempty visits	Donors
CellCNN	863	84	15	180	20
SVM	93	9	0	168	20
Citrus	1583	11	0	156	20

Table S19: Selection on the fixed 10,000-cell map and profile availability over 180 full-test-donor visits per method. The denominator for a cell is all test visits of its own donor, not 30.

Similar training-subset centroids are grouped separately by method using a shared exploratory z-scale, cosine distance, average linkage and distance cutoff 0.4. Groups occurring in at least six of 30 splits are shown. This grouping rule is a project transfer from filter-grouping code, not a verified original parameterisation of the paper’s NK-centroid analysis. To summarise marker profiles without overweighing splits with multiple members, centroids are first averaged within group/split and then summarised by the median across splits containing that group. Missing groups have no marker profile but remain in recurrence denominators.

Method	Group	Centroids	Splits	Positive	Negative
CellCNN	1	58	30	30	0
CellCNN	2	25	20	0	20
CellCNN	3	20	15	0	15
CellCNN	4	8	8	2	6
Citrus	1	28	18	18	0
Citrus	2	19	11	11	0
Citrus	3	17	10	4	7
Citrus	4	6	6	0	6
Citrus	5	7	6	6	0

Table S20: Recurring training-subset groups from the original centroid analysis. Splits count unique occurrences; positive and negative counts may overlap within a group.

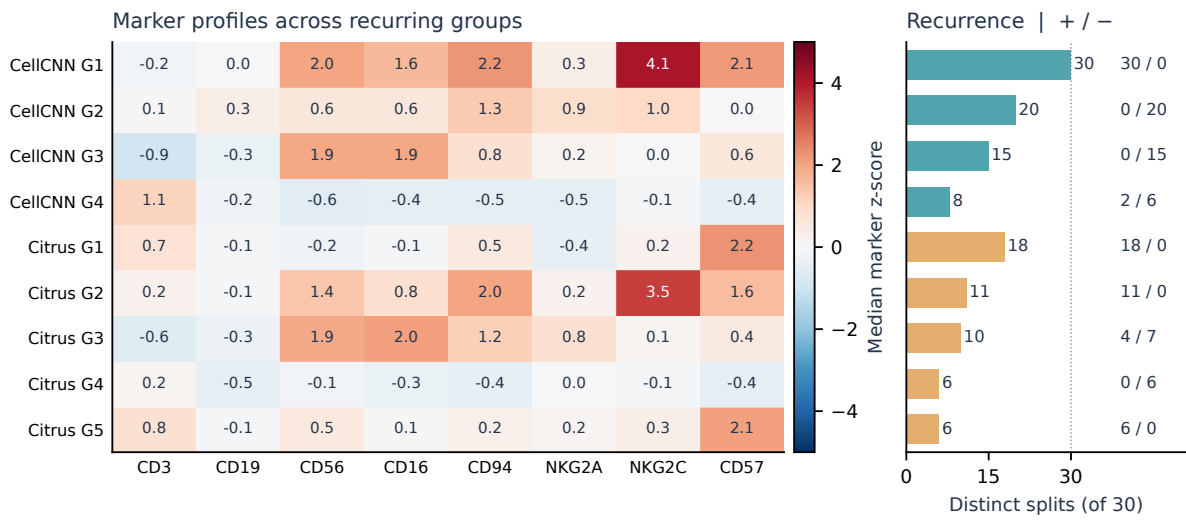


Figure S11: All retained CellCNN and Citrus centroid groups, with their marker summaries and recurrence. These are retrospective training-subset groups, not the held-out cell profiles of the main report.

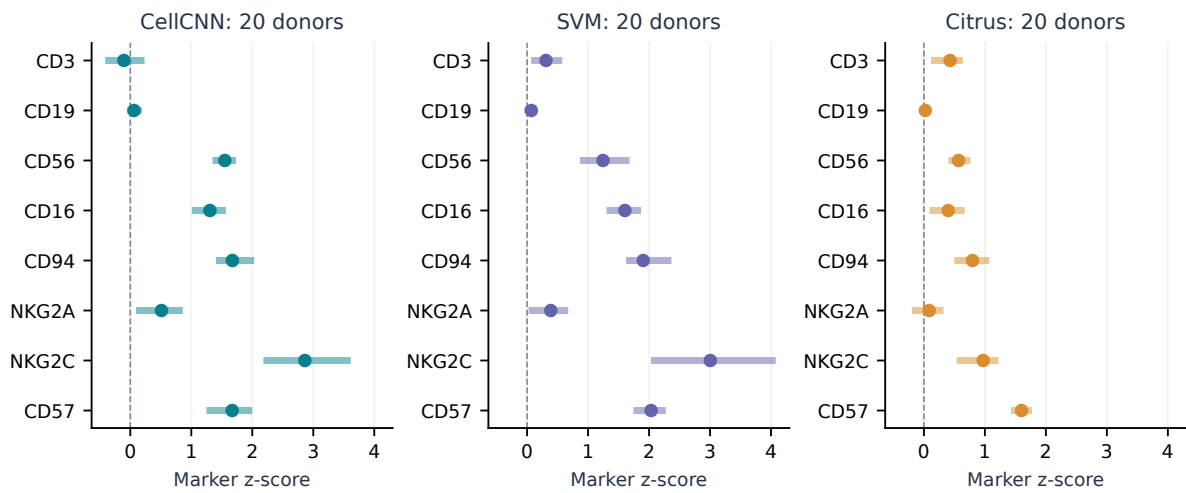


Figure S12: Held-out selected-cell profiles: dots are means across equally weighted donors; lines are Q1-Q3 of donor profiles, not confidence intervals. The contribution count is 20 donors for each method.

S5.3 Complete four-model bonus comparison

Model	Mean AUC	Median AUC	Mean AP	Mean BA
CellCNN	0.8075	0.8750	0.8038	0.6787
Quadratic Top-1 pct	0.9125	1.0000	0.9070	0.8013
Quadratic Soft-alpha	0.9125	1.0000	0.9058	0.7950
Mahalanobis Soft-alpha	0.7375	0.7500	0.7207	0.6500

Table S21: Network-score metrics over 100 shared splits. AP and BA are means; AUC includes both mean and median.

Model	Mean AUC	Median AUC	Mean effect (pp)
CellCNN	0.8559	0.8750	0.7403
Quadratic Top-1 pct	0.8610	0.9375	0.5698
Quadratic Soft-alpha	0.8559	0.9062	0.5351
Mahalanobis Soft-alpha	0.7245	0.7500	0.3004

Table S22: Filter-population frequency results over the same 98 jointly evaluable splits. Effects are mean percentage-point differences between the positive and negative test donors.

Model minus baseline	Mean delta	Median delta	Better	Equal	Worse
Quadratic Top-1 pct	0.1050	0.1250	53	36	11
Quadratic Soft-alpha	0.1050	0.1250	53	37	10
Mahalanobis Soft-alpha	-0.0700	0.0000	29	22	49

Table S23: Network-AUC differences from the CellCNN baseline over 100 splits. Equal mean AUC for the two quadratic variants is not a claim that their probabilities or every secondary metric coincide.

The quadratic soft model is better/equal/worse than the baseline in 53/37/10 splits. Its mean network-AUC improvement is 0.1050, but the corresponding mean frequency AUC does not improve over baseline. The hard quadratic variant has a slightly higher mean frequency AUC. These different summaries show why performance of the whole learned representation and performance of a single selected population must be separated.

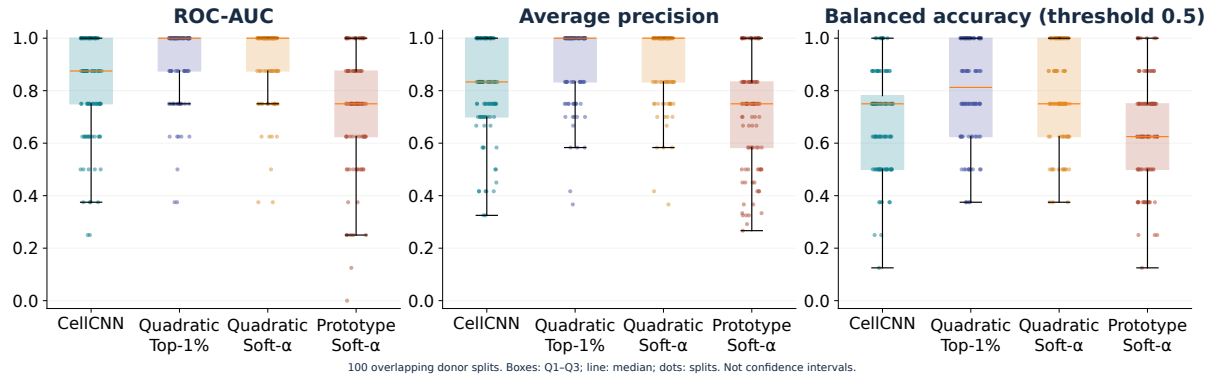


Figure S13: All four neural models on the common 100-split network benchmark.

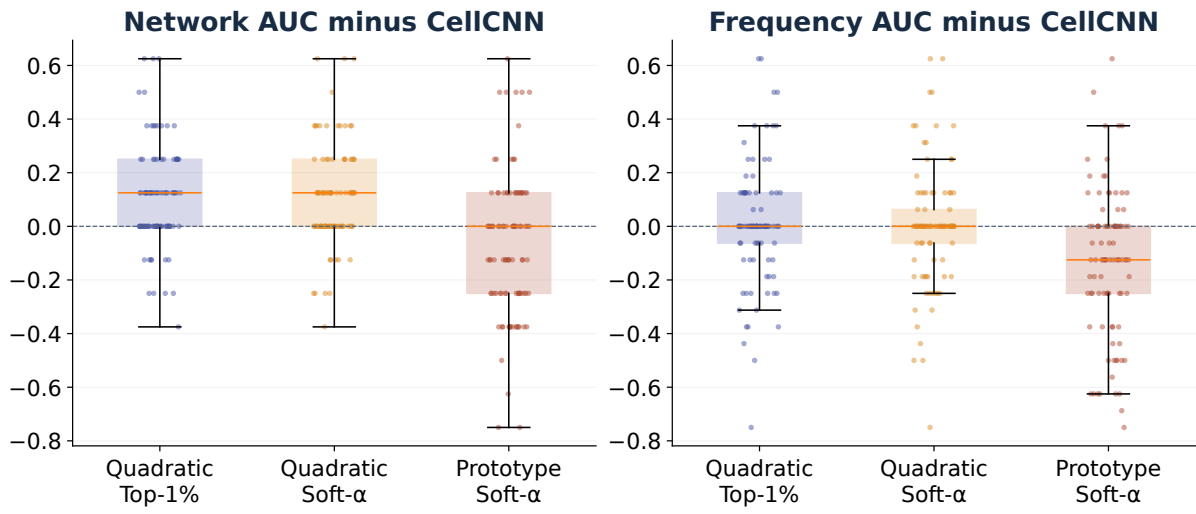


Figure S14: Paired network and population-frequency AUC differences from the baseline. The frequency panel uses only jointly evaluable splits.

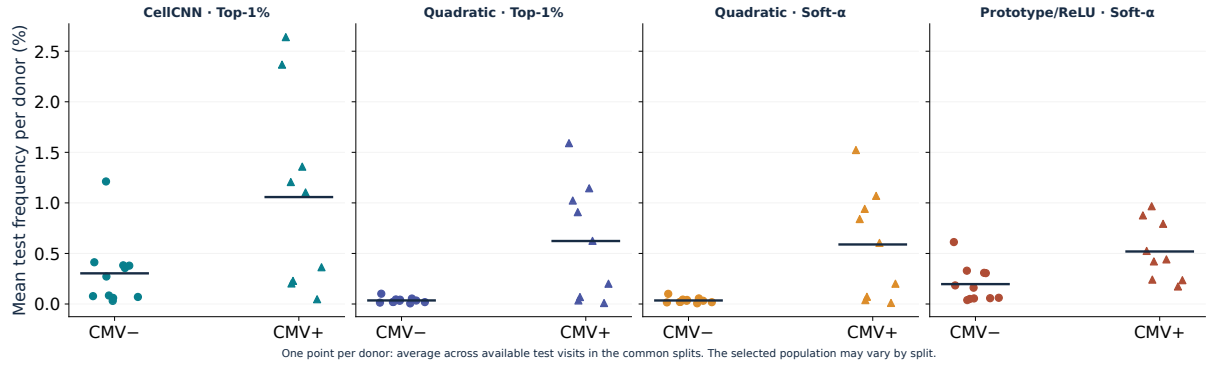


Figure S15: One point per donor, averaging its available test frequencies in the common 98 splits. The selected filter may change between splits.

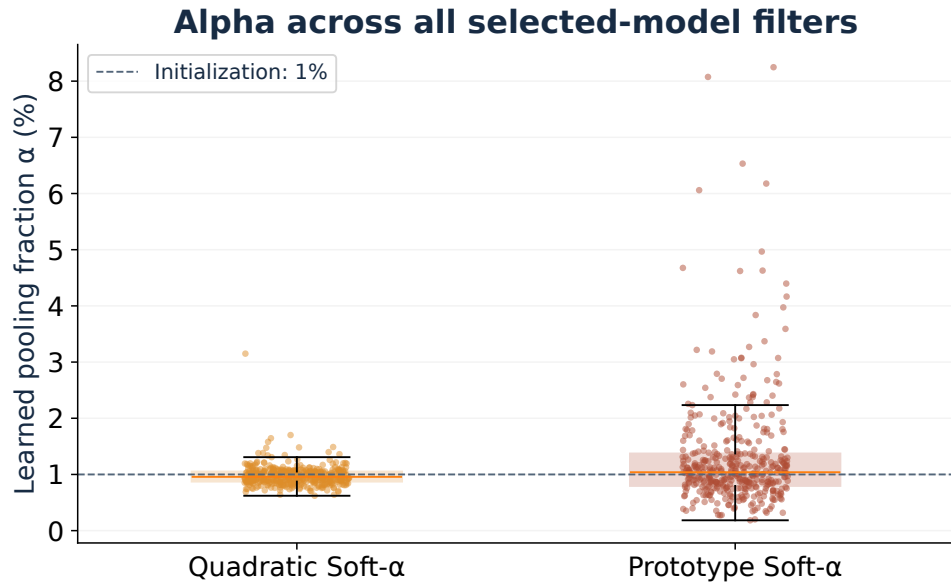


Figure S16: Learned pooling fractions over selected-model filters, with the 1% initialisation shown. The distribution characterises fitted parameters; it does not independently demonstrate a performance gain from learning the fraction.

S5.4 Result provenance and reproduction

The report uses the current executed dimensionality-reduction notebook, the original checked 20,000-cell clustering outputs, the shared 30-split classification exports, the new three-method held-out subset analysis, and the four executed 100-split bonus notebooks. Only the three selected clustering solutions were reconstructed for the new marker figures; their cluster sizes, rounded DB scores and all visible saved marker annotations agree. One existing Leiden annotation was text-truncated, so only its visible prefix could be checked. Full numeric profiles are retained in the new exports.

No dimensionality-reduction grid, resampling-stability experiment or classification benchmark was rerun to write this report. The expensive Citrus membership reconstruction belongs to the already completed subset analysis. File hashes, data scopes and the provenance of main figures are recorded alongside the report. Rebuilding the report from the supplied figure/table assets requires LaTeX, not the raw FCS archive or model training.

References

- [1] E. Arvaniti and M. Claassen. Sensitive detection of rare disease-associated cell subsets via representation learning. *Nat. Commun.* 8:14825, 2017. doi:10.1038/ncomms14825.
- [2] A. Horowitz et al. Genetic and environmental determinants of human NK cell diversity revealed by mass cytometry. *Sci. Transl. Med.* 5:208ra145, 2013. doi:10.1126/scitranslmed.3006702.
- [3] R. V. Bruggner et al. Automated identification of stratifying signatures in cellular subpopulations. *PNAS* 111:E2770–E2777, 2014. doi:10.1073/pnas.1408792111.
- [4] L. van der Maaten and G. Hinton. Visualizing data using t-SNE. *JMLR* 9:2579–2605, 2008. jmlr.org/papers/v9/vandermaten08a.html.
- [5] L. McInnes, J. Healy and J. Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018. doi:10.48550/arXiv.1802.03426.
- [6] J. Venna and S. Kaski. Neighborhood preservation in nonlinear projection methods: an experimental study. *ICANN*, pp. 485–491, 2001. doi:10.1007/3-540-44668-0_68.
- [7] D. L. Davies and D. W. Bouldin. A cluster separation measure. *IEEE TPAMI PAMI*-1:224–227, 1979. doi:10.1109/TPAMI.1979.4766909.
- [8] V. A. Traag, L. Waltman and N. J. van Eck. From Louvain to Leiden: guaranteeing well-connected communities. *Sci. Rep.* 9:5233, 2019. doi:10.1038/s41598-019-41695-z.
- [9] P. Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nat. Methods* 17:261–272, 2020. doi:10.1038/s41592-019-0686-2.
- [10] F. Pedregosa et al. Scikit-learn: machine learning in Python. *JMLR* 12:2825–2830, 2011. jmlr.org/papers/v12/pedregosa11a.html.